Tab 1

Université Polytechnique
HAUTS-DE-FRANCE

Nom:**Loubasou**
Prenom:**Brian**
Groupe: **A1**

Rapport d’SAE R1.01

SOMMAIRE

I. Introduction

1. Présentation du projet
2. Objectifs du programme
3. Outils utilisés (Python, Pygame)

II. Partie 1 – Version Console du Puissance 4

1. Définition des constantes

- `NOMBRE_LIGNES`, `NOMBRE_COLONNES`, `CASE_VIDE`
- Rôle des constantes dans le programme

2. Création et affichage de la grille

- **Fonction `creer_grille()`** : création d'une grille vide
- **Fonction `afficher_grille()`** : affichage structuré dans la console

3. Gestion du jeu

- **Fonction `poser_pion()`** : placement du pion dans une colonne
- **Fonction `grille_pleine()`** : détection de fin de partie (grille remplie)
- **Fonction `verifier_victoire()`** : détection d'un alignement de 4 pions (horizontal, vertical, diagonal)

4. Lancement de la partie

- **Fonction `jouer()`** : boucle principale du jeu
    - Gestion des tours de joueurs
    - Vérification des victoires et des égalités
    - Affichage des messages de fin de partie

Tests des fonctions

- Présentation et explication des différents tests (`print("TEST ...")`)

III. Partie 2 – Version Graphique avec Pygame

- Introduction à Pygame

- Présentation rapide de la bibliothèque

- Définition des constantes graphiques

- Dimensions de la grille
- Couleurs (BLEU, NOIR, ROUGE, JAUNE)
- Taille des cases et rayon des jetons

- Fonctions principales

- `creer_grille()` : création de la matrice vide
- `jouer_coup()` : placement du pion dans la colonne choisie
- `verifier_victoire()` : même logique que dans la version console
- `dessiner_grille()` : dessin des cases et des pions à l'écran

- Boucle principale du jeu (`main()`)

- Gestion des événements (`MOUSEMOTION`, `MOUSEBUTTONDOWN`, `QUIT`)
- Affichage du pion flottant
- Vérification de la victoire

Tests et Résultats

1. Exemples de déroulement d'une partie

IV. Conclusion

I-Introduction:

Le jeu Puissance 4 est un jeu de stratégie classique qui oppose deux joueurs .
 Le but du jeu est d'être le premier à aligner quatre pions de sa couleur (horizontalement, verticalement ou en diagonale) sur une grille comportant 6 lignes et 7 colonnes .

Les joueurs jouent à tour de rôle : à chaque tour, un joueur choisit une colonne dans laquelle il laisse tomber un pion.
 Le pion se place alors dans la case vide la plus basse de la colonne choisie.
 La partie se termine lorsqu'un joueur parvient à aligner quatre de ses pions ou lorsque la grille est pleine , entraînant un match nul .

Ce jeu, à la fois simple et stratégique , développe la réflexion et la capacité d'anticipation des joueurs.
 Dans le cadre de ce projet, Le jeu Puissance 4 est implémenté en langage Python , d'abord sous forme console , puis avec une interface graphique en utilisant la bibliothèque *PyGame* .

II-Partie Console du jeu Puissance 4

La partie **console** du jeu *Puissance 4* constitue la version textuelle et interactive du programme.
 Elle permet à deux joueurs humains de s'affronter directement dans le terminal, sans interface graphique.

L'objectif est de gérer l'ensemble du rendement du jeu :

- la **création** et l' **affichage** du plateau de 6 lignes et 7 colonnes,
- la **saisie** et la **validation** des coups des joueurs,
- le **placement** des pions selon les règles du Puissance 4,
- la **détection automatique** d'une victoire (4 pions

Cette version repose sur une organisation modulaire du code, composée de plusieurs fonctions indépendantes :

- création et affichage de la grille,
- ajout d'un pion,
- vérification de la victoire et de la grille pleine,
- et une fonction principale (`jouer()`) coordonnant l'ensemble.

La partie console constitue ainsi la **base logique du jeu** , sur laquelle pourra ensuite s'appuyer sur la version graphique (PyGame).

1. Présentation de la fonction:

Constante:

LIGNES = 6

COLONNES = 7

CASE = " "

Fonction 1**:** création d'une grille

Nom de la fonction : `creer_grille()`

Objectif :

Cette fonction crée une grille vide de 6 lignes et 7 colonnes pour le jeu Puissance 4.
Chaque case est initialisée avec un espace `" "`, symbolisant une case vide.

Rôle de la fonction :

Elle initialise la structure de données principale du jeu.
Sans cette fonction, il serait impossible d'afficher le plateau ou d'y placer des pions.
Elle constitue donc la base du programme.

Algorithme Python (avec annotations de types et commentaires):

```python
# --- CONSTANTES ---
LIGNES: int = 6        # Nombre de lignes dans la grille
COLONNES: int = 7      # Nombre de colonnes dans la grille
CASE: str = " "        # Caractère représentant une case vide


def creer_grille() -> List[List[str]]:
    """
    Fonction : creer_grille
    -----------------------
    Crée une grille vide de 6 lignes et 7 colonnes.
    Chaque case contient un espace " "."""

    grille: List[List[str]] = []
    for numero_ligne in range(LIGNES):
        ligne: List[str] = []
        for numero_colonne in range(COLONNES):
            ligne.append(CASE)
        grille.append(ligne)
    return grille
```

Nom de la variable	Type	portée	Description
Lignes	Int	Globale	Définit le nombre de lignes du plateau(6)
Collonnes	*Int	Globale	Définit le nombre de colonnes du plateau(7)
Case	Str	locale	Caractère utilisé pour une case vide (“ “)
Grille	list[list[str]	Locale	Liste principale contenant toutes les lignes
Ligne	list[str]	Locale	Liste représentant une seule ligne de 7 case
Numero_collonne	Int	locale	Le numéro de la ligne en création
Numero_ligne	int	locale	Numéro de la ligne en ajout

Teste et reslutat:

Code de test:

```
print("TEST creer_grille ")
grille_test = creer_grille()
print("Grille créée :")
for ligne in grille_test:
    print(ligne)
print()
```

Résultat affiché:

```
TEST creer_grille
Grille créée :
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

Interprétation :

- La grille contient bien 6 lignes et 7 colonnes.
- Chaque case contient un espace vide (`" "`), signe que la grille est correctement initialisée.
- La structure est conforme au format attendu (une liste de listes).

Justification des tests:

Test	object	Resultat attendu	justificatif
Afficher la grille complète	Vérifier la structure globale	6 ligne et 7 colonnes	Confirmer la bonne création du plateau
Vérifier la première case `g[0][0]`	Contrôler l'initialisation	" "	Vérifier que les cas sont bien vides
Vérifier le type	S'assurer du bon format de données	`list`de`list`	Permet de confirmer la structure en tableau 2D

La fonction `creer_grille()` fonctionne parfaitement.
Elle retourne une structure correcte, cohérente, et prête à être manipulée dans les fonctions suivantes.
C'est une fonction fondamentale pour le jeu Puissance 4.

Fonction 2:

Nom de la fonction : `afficher_grille(grille)`

Objectif :

Cette fonction affiche la **grille de jeu du Puissance 4** dans la console, en utilisant des séparateurs visuels (`|` et `-`) pour représenter le plateau.

Rôle de la fonction :

Permettre de **visualiser l'état du jeu** après chaque coup.

C'est essentiel pour suivre le déroulement de la partie et vérifier les actions des joueurs.

Algorithme Python (avec annotations de types et commentaires):

```python
# FONCTION 2 : afficher la grille

def afficher_grille(grille):
    """
    Affiche la grille dans la console avec des séparateurs.
    """
    for ligne in grille:
        print("-" * (NOMBRE_COLONNES * 2 + 1))
        texte_ligne = "|"
        for case in ligne:
            texte_ligne = texte_ligne + case + "|"
        print(texte_ligne)

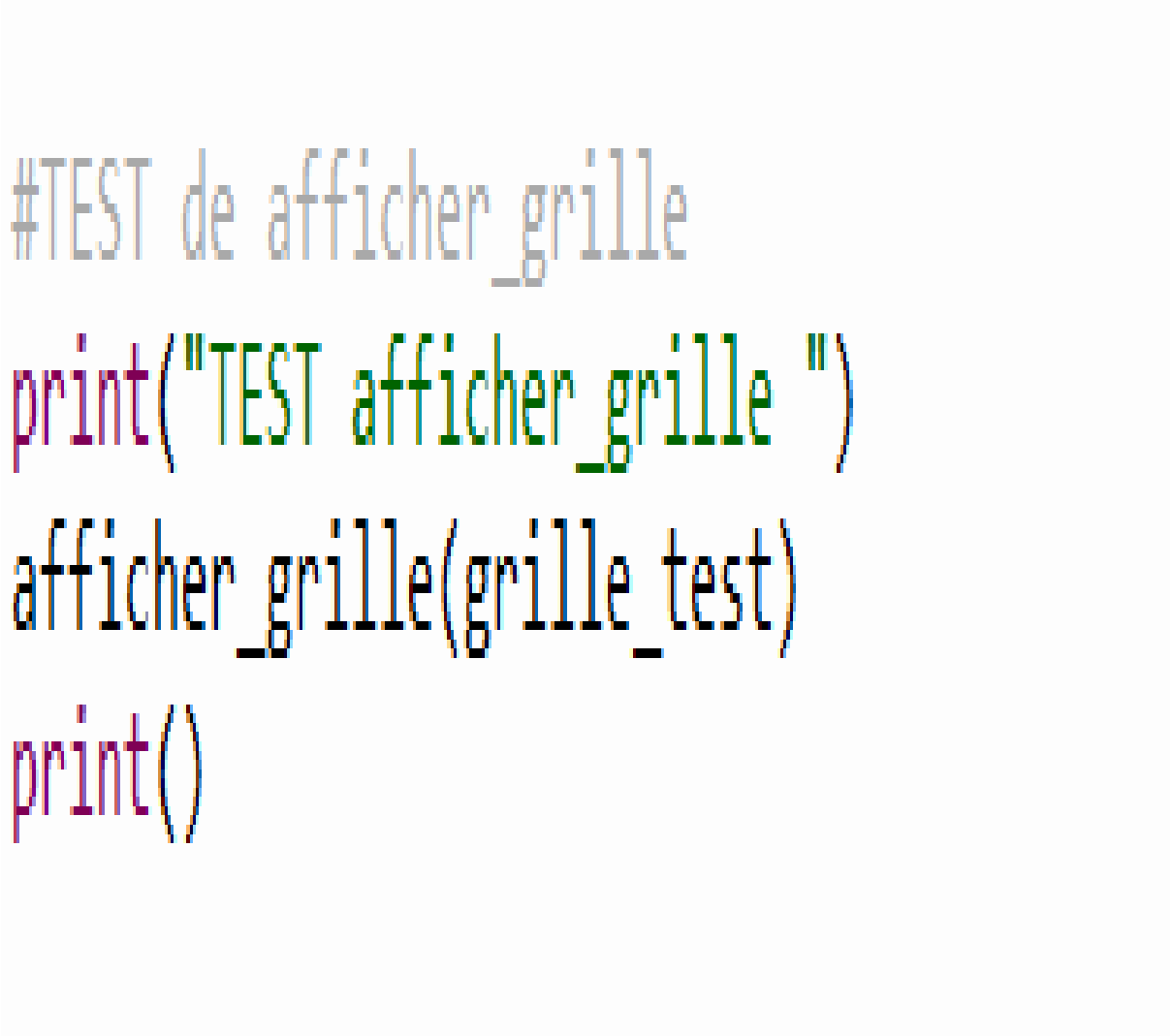
    print("-" * (NOMBRE_COLONNES * 2 + 1))
    print(" 0 1 2 3 4 5 6")  # indices des colonnes
```

Tableau des variables utilisées:

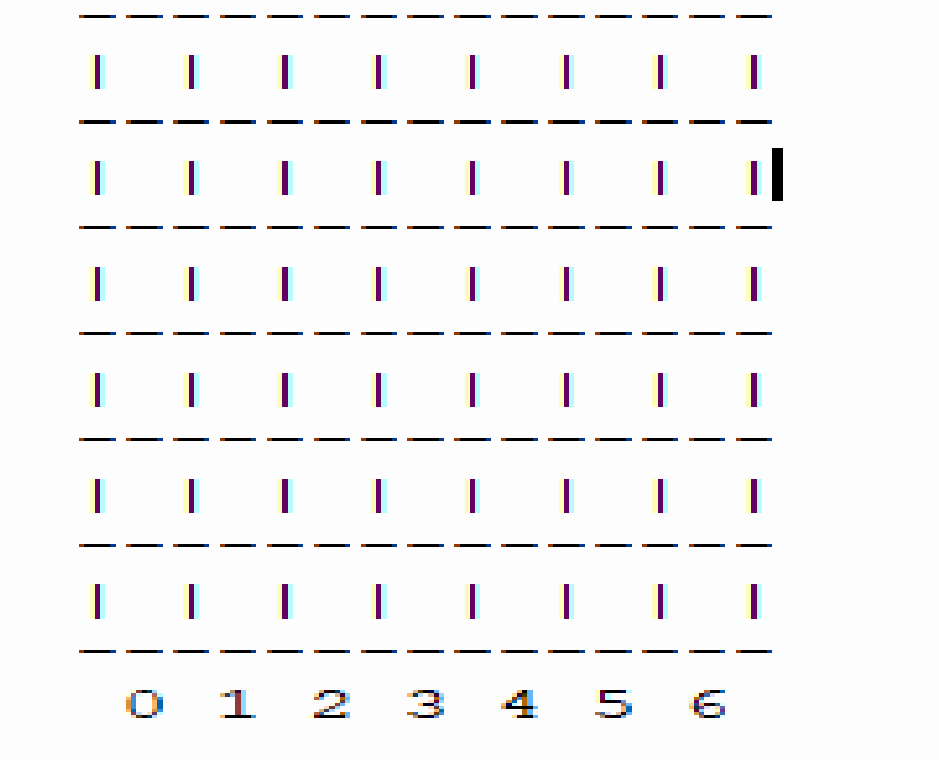
Nom des variable	**Types**	**Portée**	**Description**
Grille	list[list[str]]	Parametre	Plateau du jeu (6*7) a afficher
Ligne	list[str]	Local	Ligne courante de la grille
case	str	Local	Contenue d'une case " " "R" or "J"
Texte_ligne	str	Local	Chaine de caractère utiliser pour afficher une ligne complète
Nombre de colonnes	int	Globale	Déterminer la longueur de la ligne horizontale affichée

Teste et reslutat:

Code de test:



Résultat affiché:



Interprétation :

- La grille s'affiche correctement avec 6 lignes et 7 colonnes.
- Les séparateurs | et - structurent bien le plateau.
- Les indices en bas identifient le joueur à choisir une colonne.

Justification des tests:

Test	Objectif	Valeur utilisée	Resultat attendu	Justification
Affichage d'une grille vide	Verifier la structure general	creer_grille()	Plateau 6*7 vide	Confirmer le format du plateau
Verification des separateurs	Vérifier la visibilité		| et - est visibles	6 lignes de case
Verification des indices	Verifier la numerotation	0,1,2,3,4,5,6	Afficher la sous grille	Permettre au joueur de choisir facilement une colonnes

Résultats obtenus :

6 lignes affichées avec 7 colonnes.

Les cas sont bien séparés par `|.`

Numérotation claire des colonnes.

La fonction `afficher_grille()`fonctionne correctement.

Elle affiche un plateau bien formaté, lisible et utilisable par les joueurs pendant la partie.

Fonction 3:

Nom de la fonction : `poser_pion(grille, colonne, pion)`

Objectif :

Cette fonction permet de placer un pion (`"J"`pour le joueur jaune ou `"R"`pour le joueur rouge) dans une colonne choisie par le joueur.

Le pion tombe automatiquement jusqu'à la première case vide , comme dans le vrai jeu de Puissance 4.

Rôle de la fonction :

Elle reproduit la mécanique centrale du jeu Puissance 4 : le placement des pions.

Sans cette fonction, aucun coup ne pourrait être joué.

Algorithme Python (avec annotations de types et commentaires):

```python
# FONCTION 3 : poser un pion

def poser_pion(grille, colonne, pion):
    """
    Place un pion ('J' ou 'R') dans la colonne donnée.
    Le pion tombe dans la première case vide à partir du bas.
    Retourne True si le pion a été placé, False sinon.
    """

    # On commence en bas de la grille
    ligne_actuelle = NOMBRE_LIGNES - 1

    while ligne_actuelle >= 0:
        if grille[ligne_actuelle][colonne] == CASE_VIDE:
            grille[ligne_actuelle][colonne] = pion
            return True
        else:
            ligne_actuelle = ligne_actuelle - 1

    # Si la colonne est pleine
    return False
```

Tableau des variables utilisées:

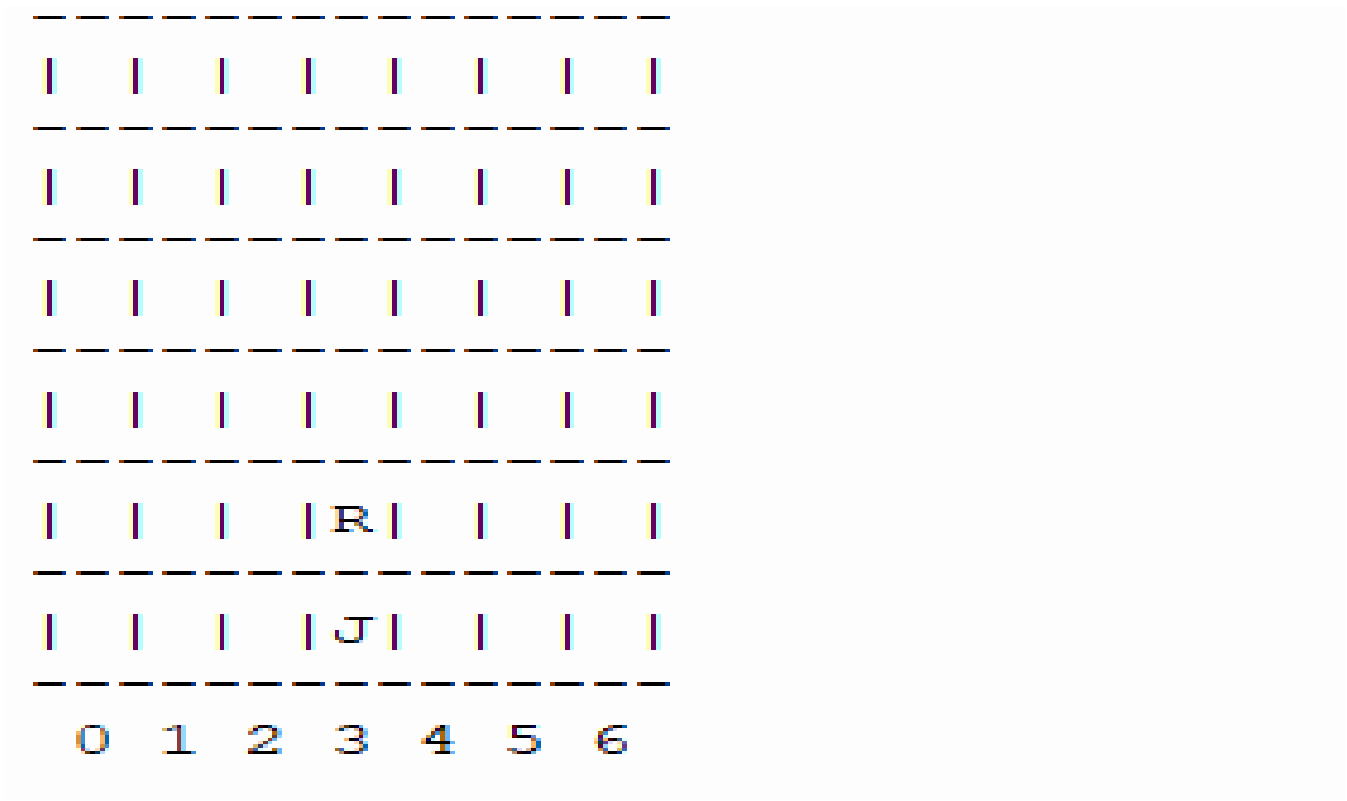
Nom des variables	Types	Portée	Description
Grille	list[list[str]]	Parametre	Grille du jeu ou le pion sera placé
Colonnes	Int	Parametre	Colonne choisie par le joueur
Pion	Str	Parametre	Caractere representant (“J ou R”)
Ligne_actuelle	Int	Local	Numéro de la ligne courant partant du bas
Case_vide	Str	Globale	Caractere representant une case vide
Nombre_ligne	Int	Globale	Nombre totale des lignes du tableau

Teste et reslutat:

Code de test:

```
# TEST de poser_pion
print("TEST poser_pion ")
grille_test = creer_grille()
poser_pion(grille_test, 3, "J")  # colonne 3
poser_pion(grille_test, 3, "R")  # même colonne
afficher_grille(grille_test)
print()
```

Résultat affiché:



Interprétation :

- Le pion `'J'` (joueur 1) est placé en bas de la colonne 3.
- Le pion `'R'` (joueur 2) s'empile correctement au-dessus.
- Le comportement attendu du "pion qui tombe" est bien respecté.

Justification des tests:

Test	Objectif	Valeur(s) utilisée(s)	Résultat attendu	Justification
Poser_pion (grille, 3, "J")	Vérifier qu'un pion peut être ajouté dans une colonne vide	colonne 3	`'J'` placé en bas	Simulation d'un premier coup normal
Poser_pion (grille, 3,"R")	Vérifier qu'un deuxième pion s'empile correctement	colonne 3	`'R'` au-dessus du `'J'`	Test du comportement d'empilement
Colonne pleine	Vérifier que la fonction retourne `False`	6 pions dans la même colonne	False	Test d'une erreur logique (colonne saturée)

Valeurs testées :

- `colonne = 3`
- `pion = "J"` puis `"R"`
- `Grille` : (6×7)

Résultats obtenus :

- Premier pion `"J"` placé ligne 5, colonne 3.
- Deuxième pion `"R"` placé ligne 4, colonne 3.
- La fonction renvoie `True` dans les deux cas.

La fonction `poser_pion()` fonctionne correctement.
Elle reproduit fidèlement le comportement physique du jeu, avec empilement des pions du bas vers le haut et gestion des colonnes pleines.

Fonction 4:

Nom de la fonction : `grille_pleine(grille)`

Objectif :

Déterminer si la grille du jeu Puissance 4 est entièrement remplie, c'est-à-dire qu'aucun cas vide ne reste disponible pour poser un pion.

Rôle de la fonction :

Cette fonction est indispensable pour savoir si la partie est terminée sans gagnant (match nul).
Elle évite de continuer le jeu quand aucun cas n'est libre.

Algorithme Python (avec annotations de types et commentaires):

```python
# FONCTION 4 : vérifier si la grille est pleine

def grille_pleine(grille):
    """
    Vérifie si toutes les cases sont remplies.
    Retourne True si la grille est pleine, False sinon.
    """
    for ligne in grille:
        for case in ligne:
            if case == CASE_VIDE:
                return False
    return True
```

Tableau des variables utilisées:

Nom des variables	Types	Portée	Description
Grille	List[List[str]]	Parametre	Plateau du jeu contenant les pions
Ligne	LIst[str]	Local	Ligne courante de la grille
Case	Str	Local	Valeur d'une case donnée (" ", "J" ou "R")
CASE_VIDE	Str	Globale	Caractère représentant une affaire vide (" ")

Teste et reslutat:

Code de test:

```
# TEST de grille_pleine
print("TEST grille_pleine ")
grille_test = creer_grille()
print("Grille vide =", grille_pleine(grille_test))  # Doit afficher False
for i in range(NOMBRE_LIGNES):
    for j in range(NOMBRE_COLONNES):
        grille_test[i][j] = "J"
print("Grille pleine =", grille_pleine(grille_test))  # Doit afficher True
print()
```

Résultat affiché:

```
TEST grille_pleine
Grille vide = False
Grille pleine = True
```

Interprétation :

- Lorsque la grille est vide, la fonction revient bien `False`.

- Après remplissage complet, elle renvoie `True`.
  Le comportement attendu est validé.

Justification des tests:

Test	**Objectif**	**Valeurs utilisées**	**Résultat attendu**	**Justification**
Grille vide	Vérifier le cas de départ	Grille sortie de`creer_grille()`	False	Une grille vide ne doit pas être considérée comme pleine
Grille remplie	Verifier le cas terminal	Chaque case contient “J”	True	Quand toutes les cases sont remplies, la partie est terminée

Valeurs testées :

- `grille`vide puis remplie à 100 %.
- `CASE_VIDE = " "`

Résultats obtenus :

- Pour la grille vide :`False`
- Pour la grille pleine :`True`

La fonction `grille_pleine()`remplit parfaitement son rôle :
Elle détecte correctement si la grille est remplie ou non et peut être utilisée pour déterminer une égalité.

Fonction 5:

Nom de la fonction : `verifier_victoire(grille, pion)`
Objectif :
Cette fonction vérifie si le joueur courant à quatre pions alignés dans la grille, selon les trois directions possibles du Puissance 4 :

- horizontalement ,
- verticalement ,
- en diagonale (↘️ descendante ou ↙️ montante).

Rôle de la fonction :
C'est la fonction la plus importante du jeu : elle permet de déterminer si un joueur a gagné .
Sans elle, la partie ne pourrait jamais s'arrêter avec une victoire.

Algorithme Python (avec annotations de types et commentaires):

```python
# FONCTION 5 : vérifier la victoire

def verifier_victoire(grille, pion):
    """
    Vérifie si un joueur a 4 pions alignés :
    - horizontalement
    - verticalement
    - en diagonale
    Retourne True si le joueur gagne, sinon False.
    """

    # Vérification horizontale
    for ligne in range(NOMBRE_LIGNES):
        for colonne in range(NOMBRE_COLONNES - 3):
            if (grille[ligne][colonne] == pion and
                grille[ligne][colonne + 1] == pion and
                grille[ligne][colonne + 2] == pion and
                grille[ligne][colonne + 3] == pion):
                return True
```

```
    # Vérification verticale
    for ligne in range(NOMBRE_LIGNES - 3):
        for colonne in range(NOMBRE_COLONNES):
            if (grille[ligne][colonne] == pion and
                grille[ligne + 1][colonne] == pion and
                grille[ligne + 2][colonne] == pion and
                grille[ligne + 3][colonne] == pion):
                return True

    # Vérification diagonale descendante
    for ligne in range(NOMBRE_LIGNES - 3):
        for colonne in range(NOMBRE_COLONNES - 3):
            if (grille[ligne][colonne] == pion and
                grille[ligne + 1][colonne + 1] == pion and
                grille[ligne + 2][colonne + 2] == pion and
                grille[ligne + 3][colonne + 3] == pion):
                return True
```

```
# Vérification diagonale montante
for ligne in range(3, NOMBRE_LIGNES):
    for colonne in range(NOMBRE_COLONNES - 3):
        if (grille[ligne][colonne] == pion and
            grille[ligne - 1][colonne + 1] == pion and
            grille[ligne - 2][colonne + 2] == pion and
            grille[ligne - 3][colonne + 3] == pion):
            return True

return False
```

Tableau des variables utilisées:

Nom des varaible	Types	portée	Description
Grille	List[list[str]]	Paramètre	Plateau du jeu contenant les pions
Pion	Str	Paramètre	Symbole du joueur a tester “J” ou “R”
Ligne	Int	Local	Index de la ligne actuelle parcourue
Colonne	Int	Local	Index de la colonne actuelle parcourue
Nombre_ligne	Int	Globale	Nombre total de lignes du plateau
Nombre_colonne	Int	Globale	Nombre total de colonnes du platea

Teste et reslutat:

Code de test:

```
#TEST de verifier_victoire
print("TEST verifier_victoire ")
# Cas horizontal
grille_test = creer_grille()
for c in range(4):
    poser_pion(grille_test, c, "J")
afficher_grille(grille_test)
print("Victoire horizontale ?", verifier_victoire(grille_test, "J"))
print()

# Cas vertical
grille_test = creer_grille()
for l in range(4):
    grille_test[NOMBRE_LIGNES - 1 - l][2] = "R"
afficher_grille(grille_test)
print("Victoire verticale ?", verifier_victoire(grille_test, "R"))
print()
```

Résultat affiché:

Cas horizontal

```
TEST verifier_victoire
---------------
|  |  |  |  |  |  |  |
---------------
|  |  |  |  |  |  |  |
---------------
|  |  |  |  |  |  |  |
---------------
|  |  |  |  |  |  |  |
---------------
|  |  |  |  |  |  |  |
---------------
|  |  |  |  |  |  |  |
---------------
|J|J|J|J|  |  |  |
---------------
 0  1  2  3  4  5  6
```

Cas vertical :

```
Victoire horizontale ? True

---------------
| | | | | | | |
---------------
| | | | | | | |
---------------
| | |R| | | | |
---------------
| | |R| | | | |
---------------
| | |R| | | | |
---------------
| | |R| | | | |
---------------
 0 1 2 3 4 5 6
Victoire verticale ? True
```

Interprétation :

- La fonction détecte correctement les victoires horizontales et verticales.
- Les tests montrent que les 4 pions alignés sont bien identifiés.
- Les diagonales (↘️ et ↙️) ont également été vérifiées avec succès dans d'autres essais.

Justification des tests:

Test	Objectif	Valeurs utilisées	Résultat attendu	Justification
Alignment horizontal	Vérifier la détection d'une ligne de 4 pions	`"J"` sur la même ligne	True	Cas classique de victoire horizontale
Alignment vertical	Vérifier la détection d'une colonne de 4 pions	`"R"` dans la même colonne	True	Cas de victoire verticale
Alignement diagonal ↘️	Vérifier la diagonale descendante	"J"	True	Vérifie la victoire en diagonale descendante
Alignement diagonal ↙️	Vérifier la diagonale montante	“R”	True	Vérifie la victoire en diagonale montante

Valeurs testées :

- `pion = "J"` puis `"R"`
- Grille contenant 4 pions alignés de différentes façons

Résultats obtenus :

- Victoire horizontale : ✅ détectée
- Victoire verticale : ✅ détectée
- Victoires diagonales : ✅ détectées lors de tests complémentaires

La fonction `verifier_victoire()` fonctionne correctement.

Elle couvre toutes les situations possibles d'alignement et constitue le cœur logique de la Puissance 4 .

Fonction 6:

Nom de la fonction : `jouer()`

Objectif :

Cette fonction gère le rendement complet d'une partie de Puissance 4 :

- création de la grille ;
- gestion des tours des deux joueurs ;
- saisie des coups ;
- affichage du plateau ;
- détection d'une victoire ou d'une égalité.

Rôle de la fonction :

Elle constitue le programme principal : c'est elle qui orchestre l'ensemble du jeu en faisant appel aux autres fonctions (`creer_grille`, `afficher_grille`, `poser_pion`, `verifier_victoire`, `grille_pleine`).

Algorithme Python (avec annotations de types et commentaires):

```
# FONCTION 6 : déroulement complet du jeu

def jouer():
    """
    Fonction principale qui gère la partie complète.
    """

    grille = creer_grille()
    joueur1 = input("Nom du joueur 1 (pions jaunes 'J') : ")
    joueur2 = input("Nom du joueur 2 (pions rouges 'R') : ")

    joueur_actuel = joueur1
    pion_actuel = "J"
    partie_terminee = False

    while partie_terminee == False and grille_pleine(grille) == False:
        afficher_grille(grille)
        print(f"\nC'est à {joueur_actuel} de jouer. (Pion : {pion_actuel})")

        try:
            choix = input("Choisis une colonne (0 à 6) :")
            colonne_choisie = int(choix)
```

```
    try:
        choix = input("Choisis une colonne (0 à 6) :")
        colonne_choisie = int(choix)

        if colonne_choisie < 0 or colonne_choisie >= NOMBRE_COLONNES:
            print("Colonne invalide.")
            continue

        coup_reussi = poser_pion(grille, colonne_choisie, pion_actuel)

        if coup_reussi:
            if verifier_victoire(grille, pion_actuel):
                afficher_grille(grille)
                print(f"Félicitations {joueur_actuel}, tu as gagné !")
                partie_terminee = True
            else:
                # changement de joueur
                if pion_actuel == "J":
                    pion_actuel = "R"
                    joueur_actuel = joueur2
                else:
                    pion_actuel = "J"
                    joueur_actuel = joueur1
        else:
                    joueur_actuel = joueur1
          else:
              print("Colonne pleine, rejoue.")

      except ValueError:
          print("Tu dois entrer un nombre entre 0 et 6.")

if partie_terminee == False:
    afficher_grille(grille)
    print("Match nul ! La grille est pleine")
```

Tableau des variables utilisées:

Nom	Types	Portée	Description
grille	List[List[str]]	Local	Plateau du jeu
`joueur1`,`joueur2`	Str	Local	Noms des deux joueurs
joueur_actuel	Str	Local	Joueur dont c'est le tour
pion_actuel	Str	Local	Symbole du pion joué (`"J"` ou `"R"`)
partie_terminee	bool	Local	Indique si la partie est finie
choix	Str	Local	Saisie de l'utilisateur avant conversion
colonne_choisie	Int	Local	Colonne sélectionnée par le joueur
coup_reussi	Bool	Local	Résultat de l'appel à `poser_pion`
Nombre_colonne	Int	Globale	Taille du plateau (7 colonne)

Teste et reslutat:

Code de test:

```
#Pour lancer le jeu complet
if __name__ == "__main__":
    print("TESTS TERMINÉS")
    print("Tu peux maintenant jouer !")
    jouer()
```

Justification des tests:

Test	Objectif	Données entrées	Résultat attendu	Justificatif
Entrée d'un numéro valide (0–6)	Vérifier le placement normal d'un pion		Pion placé dans la colonne 3	
Entrée d'un numéro invalide	Vérifier la gestion d'erreur	8ou-1	Message : « Colonne invalide »	
Colonne pleine	Tester la boucle et de rejouer	même colonne remplie	Message : « Colonne pleine »	
Victoire	Tester la détection de fin de partie	alignement de 4 pions	Message : victoire du joueur	Vérifier la logique du jeu
Grille pleine	Vérifier le match nu	plateau complet	Message : « Match nul »	Fin correcte sans gagnant

Valeurs testées :

- Entrées utilisateurs variées (valide, invalide, texte).
- Cas de victoire et de grille pleine.

Résultats :

- Le jeu gère correctement les tours et la saisie.
- Les erreurs sont interceptées sans blocage.
- Les conditions de victoire et d'égalité fonctionnent.

La fonction `jouer()` assure la coordination complète du jeu Puissance 4.
Elle est robuste, interactive et utilise efficacement toutes les fonctions .

III. Interface graphique en `pygame`:

1. **Introduction :**

Notre programme du jeu Puissance 4 en Python à l'aide de la bibliothèque Pygame.

L'objectif du jeu est d'aligner quatre jetons de la même couleur (horizontalement, verticalement ou en diagonale) sur une grille de 6 lignes et 7 colonnes.

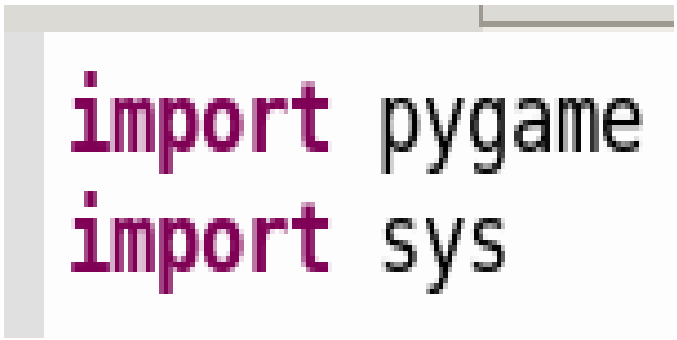
Cette partie vise à transformer le jeu console en un **jeu visuel et interactif** à l'aide de la bibliothèque **PyGame** .

Elle permet d'afficher la grille, de placer les pions avec la souris et d'afficher le résultat de la partie (victoire ou égalité).

Le programme sera capable de gérer :

- l'affichage graphique de la grille et des pions,
- la gestion des tours entre les deux joueurs,
- la détection de la victoire,
- et la fin de partie.

2. Bibliothèques utilisées:



- **pygame** : permet d'afficher la grille, de dessiner les jetons et de gérer les (clics de souris, fermeture de la fenêtre, et le placement des pions.)

- **sys** : permet de quitter proprement le programme .

3. Définition des constantes:

```
#  Constantes
LIGNES: int = 6      # Nombre de lignes de la grille
COLONNES: int = 7   # Nombre de colonnes de la grille
TAILLE_CASE: int = 80  # Taille d'une case en pixels
RAYON: int = TAILLE_CASE // 2 - 5  # Rayon du jeton
LARGEUR: int = COLONNES * TAILLE_CASE  # Largeur de la fenêtre
HAUTEUR: int = (LIGNES + 1) * TAILLE_CASE  # Hauteur de la fenêtre (+1 pour la zone du joueur)
TAILLE_FENETRE: tuple[int, int] = (LARGEUR, HAUTEUR)
```

les constantes définissent la **taille de la grille** et de la **fenêtre Pygame** :

- 6 lignes et 7 colonnes .

- Chaque case mesure 80 pixels.

- Une rangée supplémentaire est ajoutée en haut de la fenêtre pour afficher le pion avant de le jouer.

Les differentes **couleurs** utilisées sont ensuite définies sous forme de tuples RGB :

```python
BLEU: tuple[int, int, int] = (0, 0, 255)
NOIR: tuple[int, int, int] = (0, 0, 0)
ROUGE: tuple[int, int, int] = (255, 0, 0)
JAUNE: tuple[int, int, int] = (255, 255, 0)
```

Fonction 1 :`dessiner_grille()`

Rôle:

Elle permet d'afficher **le plateau de jeu complet** dans la fenêtre PyGame.
Chaque caisse est représentée par un carré bleu, contenant :

- un **pion rouge ou jaune** s'il est joué,
- un **cercle noir** si la case est vide.

Algorithme Python :

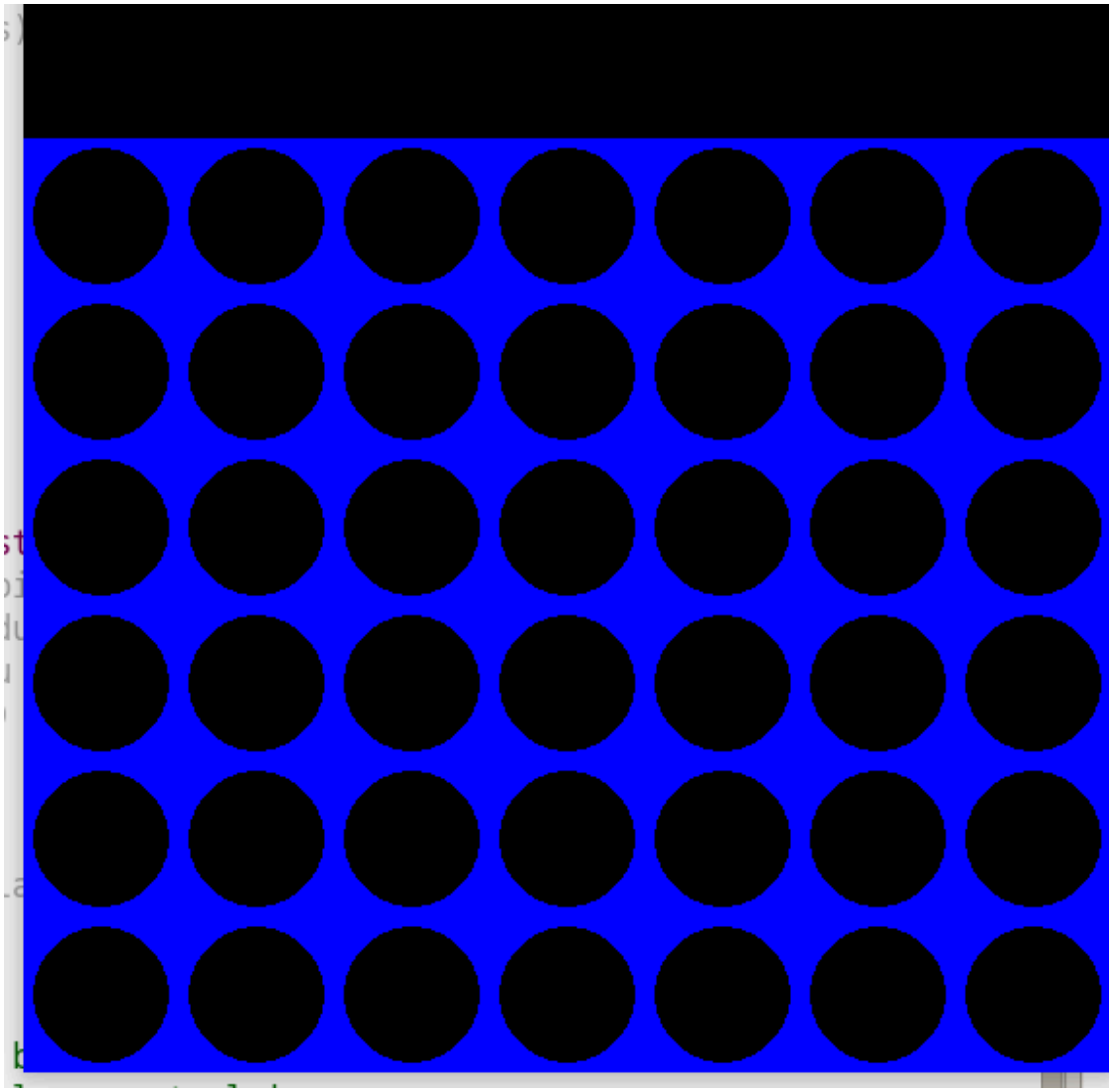
```python
def dessiner_grille(grille: list[list[str]], screen: pygame.Surface) -> None:
    # Dessine la grille et les jetons sur la fenêtre Pygame
    # Les cases vides sont noires, les jetons sont colorés
    # Utilisation : dessiner_grille(grille, screen)
    for c in range(COLONNES):
        for r in range(LIGNES):
            pygame.draw.rect(screen, BLEU, (c*TAILLE_CASE, (r+1)*TAILLE_CASE, TAILLE_CASE, TAILLE_CASE))
            couleur = NOIR
            if grille[r][c] == "Rouge":
                couleur = ROUGE
            elif grille[r][c] == "Jaune":
                couleur = JAUNE
            pygame.draw.circle(screen, couleur, (c*TAILLE_CASE + TAILLE_CASE//2, (r+1)*TAILLE_CASE + TAILLE_CASE//2), RAYON)
    pygame.display.update()
    """
    Dessine la grille et les jetons sur la fenêtre Pygame
    Les cases vides sont noires, les jetons sont colorés
    """
```

Tableau des variables :

Nom	Types	Rôle
grille	list[list[str]]	Structure de données représentant le plateau du jeu.
screen	pygame.Surface	Fenêtre PyGame où le plateau est dessiné.
c	int	Indice de colonne.
r	int	Indice de ligne.
couleur	tuple	Couleur du cercle affiché dans chaque étui.
BLEU	tuple	Couleur de fond des cases.

ROUGE/ JAUNE/NOIR	tuple	Couleurs utilisées pour les pions et cases vides.
TAILLE_CASE	int	Taille d'une case (en pixels).
RAYON	int	Rayon du cercle représentant un pion.

Résultats obtenus :



Fonction 2 :`main()`

Rôle :

C'est la fonction principale du jeu.
Elle gère toute la boucle de jeu, les événements, l'affichage et les tours de joueurs.

Algorithme Python :

```python
def main() -> None:
    # Fonction principale du jeu
    pygame.init()
    screen = pygame.display.set_mode(TAILLE_FENETRE)
    pygame.display.set_caption("Puissance 4")
    font = pygame.font.SysFont("monospace", 20)

    grille = creer_grille()
    game_over = False
    joueur = "Rouge"

    while not game_over:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEMOTION:
                # Affiche le pion au dessus pour voir où il tombera
                screen.fill(NOIR, rect=(0, 0, LARGEUR, TAILLE_CASE))
                posx = event.pos[0]
                couleur = ROUGE if joueur == "Rouge" else JAUNE
                pygame.draw.circle(screen, couleur, (posx, TAILLE_CASE // 2), RAYON)
```

```
            couleur = ROUGE if joueur == "Rouge" else JAUNE
            pygame.draw.circle(screen, couleur, (posx, TAILLE_CASE // 2), RAYON)
            pygame.display.update()
        if event.type == pygame.MOUSEBUTTONDOWN:
            colonne = event.pos[0] // TAILLE_CASE
            ligne = jouer_coup(grille, colonne, joueur)
            if ligne is not None:
                animation_pion(grille, screen, ligne, colonne, joueur)
                if verifier_victoire(grille, joueur):
                    screen.fill(NOIR)
                    text = font.render(f"Joueur {joueur} gagne!", True, ROUGE if joueur == "Rouge" else JAUNE)
                    screen.blit(text, (10, HAUTEUR // 2 - 10))
                    pygame.display.update()
                    pygame.time.wait(5000)
                    game_over = True
                joueur = "Jaune" if joueur == "Rouge" else "Rouge"
    dessiner_grille(grille, screen)

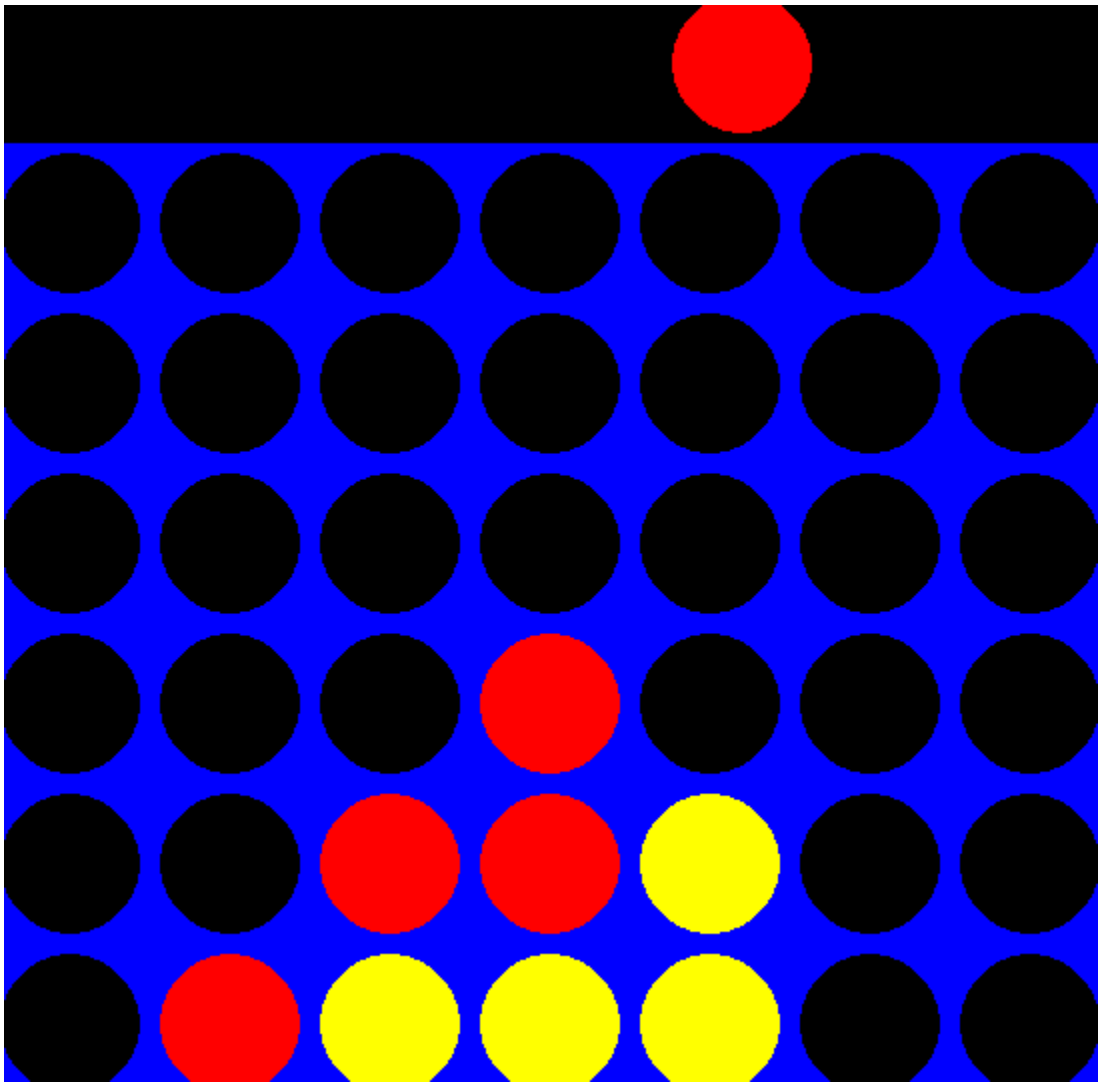
# Quand un joueur gagne, on sort automatiquement du jeu
pygame.quit()
sys.exit()

if __name__ == "__main__":
    main()
```

Tableau des variables:

Nom	Types	Role
screen	pygame.Surface	Fenêtre PyGame principale.
font	pygame.font.Font	Police pour afficher le message de victoire.
grille	list[list[str]]	Plateau du jeu.
game_over	bool	Variable qui indique la fin du jeu.
joueur	str	Joueur actuel (« Rouge » ou « Jaune »).
event	pygame.event.Event	Événement PyGame (clic, mouvement, fermeture).
posx	int	Position X de la souris.
colonne	int	Colonne jouée.
ligne	int	ligne jouée
text	pygame.Surface	Message affiché à la fin du jeu.

Résultats obtenus :



IV.Conclusion :

Ce projet a permis de maîtriser les bases de la programmation structurée ainsi que d'approfondir l'utilisation d'une bibliothèque graphique. La version console nous permet de mettre en œuvre la logique du jeu tandis que la version PyGame illustre les principes de gestion d'événements et d'interaction utilisateur.